ERDP Architecture

<table>
<tr><td colspan="4">ERDP Architecture</td></tr>
<tr><td>Application</td><td colspan="2">APP</td><td rowspan="6">C\C++</td></tr>
<tr><td>Service</td><td colspan="2">Service</td></tr>
<tr><td>3rd-party module</td><td>Lib</td><td rowspan="2"></td></tr>
<tr><td>BSP</td><td colspan="2">Board</td></tr>
<tr><td>Abstract Layer</td><td>OSAL</td><td>HAL</td></tr>
<tr><td>Interface Layer</td><td>OSIF</td><td>HIF</td><td rowspan="2">C</td></tr>
<tr><td>Kernel code</td><td>RTOS</td><td>Driver</td></tr>
</table>

1. 设计目标

ERDP 采用分层架构，目标是将业务逻辑与芯片细节解耦，实现：

- 跨芯片平台复用（当前覆盖 `STM32F4XX` 和 `GD32F4XX`）。
- 业务代码面向统一 API 编程，不直接依赖寄存器或厂商 SDK。
- 构建系统可按芯片/调试器切换，快速生成可下载镜像。

---

2. 分层结构（自上而下）

Application（示例层）

- 路径：`Source/Application`

- 作用：演示线程、日志、外设调用方式。
- 说明：`main` / `app` 仅作为 demo 入口，不承载架构核心逻辑。

Service（服务层）

- 路径：`Source/Service`（当前主要是 `Log`）
- 作用：提供通用服务能力（如日志聚合与输出）。
- 特点：依赖下层抽象，不直接碰芯片寄存器。

Library（功能库层）

- 路径：`Source/Library`（如 `st7789`）
- 作用：封装可复用功能组件，按统一 HAL/OSAL 接口工作。

HAL（硬件抽象层）

- 路径：`Source/HAL`
- 作用：提供 C++ 风格设备对象（GPIO/UART/SPI 等），对上暴露稳定 API。
- 特点：不直接绑定具体芯片，通过 Interface 层落地到平台实现。

OSAL（操作系统抽象层）

- 路径：`Source/OSAL`
- 作用：统一线程、事件、定时器、内存等 OS 语义。
- 特点：将 FreeRTOS 等 RTOS 差异封装到统一接口，供上层透明使用。

Interface（接口适配层）

- 路径：`Source/Interface`
- 子层：
  - `Drivers`：GPIO/UART/SPI/I2C/EXTI 等外设适配。
  - `RTOS`：任务调度/同步原语适配。
- 作用：承接 HAL/OSAL 的抽象调用，并映射到具体平台驱动。

Kernel（内核与底层驱动层）

- 路径：`Source/Kernel`

- 子层：
  - `Drivers`：芯片厂商外设驱动/CMSIS/启动文件/链接脚本。
  - `RTOS`：FreeRTOS 内核与移植端口。
- 作用：提供最底层执行与硬件访问能力。

Board（板级配置层）

- 路径：`Source/Board`
- 作用：定义板级资源映射与编译配置（例如 UART/SPI 引脚、板型选择）。
- 特点：平台差异通过配置汇聚，避免上层散落条件编译。

---

3. 依赖与调用方向

核心原则：**仅允许上层依赖下层，不反向依赖。**

典型调用链：

`Application -> Service/Library -> HAL + OSAL -> Interface -> Kernel(Drivers/RTOS)`

这样可以保证：

- 替换芯片平台时，主要修改 `Interface/Kernel/Board`。
- 业务代码和大部分中间层无需改动。

---

4. 平台可移植机制

项目通过以下方式实现平台切换：

- `CMakePresets.json` 选择芯片、驱动路径、启动文件、链接脚本。
- 顶层 `CMakeLists.txt` 根据 `DRIVER_PATH` 推导 `CHIP_DIR`，自动引入对应 include 与定义。
- `Source/Interface/Drivers/STM32F4XX` 与 `Source/Interface/Drivers/GD32F4XX` 提供同名接口实现。

因此，上层代码使用同一套 ERDP API，即可在 STM32/GD32 间切换。

5. 构建产物与调试关系

- 目标产物：`app.elf`、`app.hex`。
- `build/flash/flash.elf` 用于统一调试/烧录入口。
- 架构层次与构建模块一一对应（`Kernel/Interface/HAL/OSAL/Library/Service/Application`），便于模块化编译与定位问题。

---

6. 总结

ERDP 的本质是“**分层抽象 + 平台适配**”：

- 上层聚焦业务逻辑；
- 中层提供统一能力接口；
- 下层吸收芯片与 RTOS 差异。

这种设计适合在同类 MCU 项目中长期复用与演进。